



7

Handling Navigation with Routes

Almost every web application requires routing, which is the process of responding to a URL based on a set of route handler declarations. In other words, this is a mapping from the URL to rendered content. However, this task is more involved than it seems at first, due to the complexities of managing different URL patterns and mapping them to appropriate content rendering. This includes handling nested routes and dynamic parameters and ensuring proper navigation flow. The complexities of these tasks are why you're going to leverage the `react-router` package in this chapter, the de facto routing tool for React.

First, you'll learn the basics of declaring routes using JSX syntax. Then, you'll learn about the dynamic aspects of routing, such as dynamic path segments and query parameters. Next, you'll implement links using components from `react-router`.

Here are the high-level topics that we'll cover in this chapter:

- Declaring routes
- Handling route parameters
- Using link components

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter07>.

Declaring routes

With `react-router`, you can collocate routes with the content that they render. By defining routes using JSX syntax alongside the components they are associated with, `react-router` empowers developers to create a clear and logical structure for their React applications. This collocation makes it easier to understand how different parts of the application are connected and navigated, leading to improved readability and maintainability of the code base.

Throughout this chapter, we'll explore the fundamentals of routing in React applications using `react-router`. We'll start by creating a basic example route to familiarize ourselves with the syntax and structure of route declarations. Then, we'll dive deeper into organizing routes by feature, rather than relying on a monolithic routing module. Finally, we'll implement a

common parent-child routing pattern to demonstrate how to handle more complex routing scenarios.

Hello route

Before we start writing code, let's set up the `react-router` project. Run the following command to add `react-router-dom` to the dependencies:

```
npm install react-router-dom
```

Let's create a simple route that renders a simple component:

1. First, we have a small React component that we want to render when the route is activated:

```
function MyComponent() {  
  return <p>Hello Route!</p>;  
}
```

2. Next, let's look at the route definition:

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import { createBrowserRouter, RouterProvider } from "react-router-dom";  
import MyComponent from "./MyComponent";
```

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <MyComponent />,
  },
]);
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

The `RouterProvider` component is the top-level component of the application. Let's break it down to find out what's happening within the router.

You have the actual routes declared in the `createBrowserRouter` function. There are two key properties of any route: `path` and `element`. When the `path` property is matched against the active URL, the component is rendered. But where is it rendered, exactly? The router doesn't actually render anything itself; it's responsible for managing how other components are connected based on the current URL. In other words, the router checks the current URL and returns the corresponding component from the `createBrowserRouter` declaration. Sure enough, when you look at this example in a browser, `<MyComponent>` is rendered as expected:

Hello Route!

Figure 7.1: The rendered output of our component

When the `path` property matches the current URL, the route component is replaced by the `element` property value. In this example, the route returns `<MyComponent>`. If a given route doesn't match, nothing is rendered.

This example shows the fundamentals of routing in React. It's really simple and intuitive to declare routes. To further solidify your understanding of `react-router`, I encourage you to experiment with the concepts we've covered. Try creating more routes on your own, and observe how they impact the behavior of your application. After that, you can try more advanced techniques like lazy loading components using `React.lazy` and `Suspense` (you'll learn more about these in the next chapter), and implement route-based code splitting to optimize your application's performance. By diving deeper into these topics and applying them to your own projects, you'll gain a greater appreciation for the capabilities of `react-router` and its role in building modern, efficient, and user-friendly React applications.

Decoupling route declarations

The difficulty with routing happens when your application has dozens of routes declared within a single module since it's more difficult to mentally map routes to features.

To help with this, each top-level feature of the application can define its own routes. This way, it's clear which routes belong to which feature. So, let's start with the `App` component:

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <Layout />,
    children: [
      {
        index: true,
        element: <h1>Nesting Routes</h1>,
      },
      routeOne,
      routeTwo,
    ],
  },
]);
export const App = () => <RouterProvider router={router} />;
```

In this example, the application has two routes: `one` and `two`. These are imported as route objects and placed inside `createBrowserRouter`. The first `element` in this router is the `<Layout />` component, which renders a page template with

data that never changes and serves as a place for our route data. Let's take a look at the `<Layout />` component:

```
function Layout() {  
  return (  
    <main>  
      <nav>  
        <Link to="/">Main</Link>  
        <span> | </span>  
        <Link to="/one">One</Link>  
        <span> | </span>  
        <Link to="/two">Two</Link>  
      </nav>  
      <Outlet />  
    </main>  
  );  
}
```

This component contains a small navigation toolbar with links and the `<Outlet />` component. It's a built-in `react-router` component that will be replaced with matched route elements.

The router only gets as big as the number of application features, instead of the number of routes, which could be substantially larger. Let's take a look at one of the feature routes:

```
const routes: RouteObject = {  
  path: "/one",
```

```
element: <Outlet />,
children: [
  {
    index: true,
    element: <Redirect path="/one/1" />,
  },
  {
    path: "1",
    element: <First />,
  },
  {
    path: "2",
    element: <Second />,
  },
],
};
```

This module, `one/index.js`, exports a configuration object with three routes:

- When the `/one` path is matched, redirect to `/one/1`.
- When the `/one/1` path is matched, render the `First` component.
- When the `/one/2` path is matched, render the `Second` component.

This means that when the app loads the URL, `/one`, the `<Redirect>` component, will send the user to `/one/1`. Like

the `RouterProvider`, the `Redirect` component lacks UI elements inside; it solely manages logic.

This aligns with React's practice of embedding components in layouts to handle specific functionalities. This approach allows for a clean separation of concerns, with components focused solely on rendering UI elements and others, like `Redirect`, dedicated to handling routing logic. The `Redirect` component in `react-router` is responsible for programmatically navigating the user to a different route. It's commonly used to redirect users from one URL to another based on certain conditions, such as authentication status or route parameters. By abstracting away the navigation logic into a separate component, it promotes code reusability and maintainability within the application.

You're using `Redirect` here because we don't have content on the root route. Often, your application doesn't actually have content to render at the root of a feature, or at the root of the application itself. This pattern allows you to send the user to the appropriate route and the appropriate content. Here's what you'll see when you open the app and click on the **One** link:

Main | One | Two

Feature 1, page 1

Figure 7.2: The contents of page 1

The second feature follows the exact same pattern as the first. Here's what the `First` component looks like:

```
export default function First() {  
  return <p>Feature 1, page 1</p>;  
}
```

Each feature, in this example, uses the same minimal rendered content. These components are ultimately what the user needs to see when they navigate to a given route. By organizing routes this way, you've made your features self-contained with regard to routing.

In the following section, you'll learn how to further organize your routes into parent-child relationships.

Handling route parameters

The URLs that you've seen so far in this chapter have all been static. Most applications will use both **static** and **dynamic** routes. In this section, you'll learn how to pass dynamic URL segments to your components, how to make these segments optional, and how to get query string parameters.

Resource IDs in routes

One common use case is to make the ID of a resource part of the URL. This makes it easy for your code to get the ID and then make an **API** call that fetches the relevant resource data. Let's implement a route that renders a user detail page. This will require a route that includes the user ID, which then needs to somehow be passed to the component so that it can fetch the user.

Let's start with the `App` component that declares the routes:

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <UsersContainer />,
    errorElement: <p>Route not found</p>,
  },
  {
    path: "/users/:id",
    element: <UserContainer />,
  },
])
```

```
errorElement: <p>User not found</p>,
loader: async ({ params }) => {
  const user = await fetchUser(Number(params.id));
  return { user };
},
]);
function App() {
  return <RouterProvider router={router} />;
}
```

The `:` syntax marks the beginning of a URL variable. The `id` variable will be passed to the `UserContainer` component. Before displaying the component, the `loader` function is triggered, asynchronously fetching data for the specified user ID. In the case of data loading errors, the `errorElement` prop provides a fallback to handle such situations effectively. Here's how `UserContainer` is implemented:

```
function UserContainer() {
  const params = useParams();
  const { user } = useLoaderData() as { user: User };
  return (
    <div>
      User ID: {params.id}
      <UserData user={user} />
    </div>
  );
}
```

```
);  
}
```

The `useParams()` hook is used to get any dynamic parts of the URL. In this case, you're interested in the `id` parameter. Then, we get `user` from the loader function using the `useLoaderData` hook. If the URL is missing the segment completely, then this code won't run at all; the router will revert us to the `errorElement` component.

Now, let's take a look at the API functions that were used in this example:

```
export type User = {  
  first: string;  
  last: string;  
  age: number;  
};  
const users: User[] = [  
  { first: "John", last: "Snow", age: 40 },  
  { first: "Peter", last: "Parker", age: 30 },  
];  
export function fetchUsers(): Promise<User[]> {  
  return new Promise((resolve) => {  
    resolve(users);  
  });  
}  
export function fetchUser(id: number): Promise<User> {
```

```
return new Promise((resolve, reject) => {
  const user = users[id];
  if (user === undefined) {
    reject('User ${id} not found');
  } else {
    resolve(user);
  }
});
}
```

The `fetchUsers()` function is used by the `UsersContainer` component to populate the list of user links. The `fetchUser()` function will find and resolve a value from the `users` array of the mock data.

Here is the `User` component, which is responsible for rendering the user details:

```
type UserDataProps = {
  user: User;
};
function UserData({ user }: UserDataProps) {
  return (
    <section>
      <p>{user.first}</p>
      <p>{user.last}</p>
      <p>{user.age}</p>
    </section>
  );
}
```

```
);  
}
```

When you run this app and navigate to `/`, you should see a list of users that looks like this:

- [John](#)
- [Peter](#)

Figure 7.3: The contents of the app home page

Clicking on the first link should take you to `/users/0`, which looks like this:

User ID: 0

John

Snow

40

Figure 7.4: The contents of the user page

If you navigate to a user that doesn't exist, for example, `/users/2`, here's what you'll see:

User not found

Figure 7.5: When a user isn't found

The reason that you get this error message instead of a 500 error is that the API endpoint knows how to deal with missing resources:

```
if (user === undefined) {  
  reject('User ${id} not found');  
}
```

This rejection will be handled by `react-router` with the provided `errorElement` component.

In the next section, we'll look at defining optional route parameters.

Query parameters

Sometimes, we need optional URL path values or query parameters. URLs work best for simple options, and query parameters work best if there are many values that the component can use.

Let's implement a user list component that renders a list of users. Optionally, you want to be able to sort the list in descending order. Let's make this with the route that can accept a query string:

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <UsersContainer />,
  },
]);
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

There is no special setup in the router for handling query parameters. It's up to the component to handle any query strings provided to it. So, while the route declaration doesn't provide a mechanism to define accepted query strings, the router will

still pass the query parameters to the component. Let's take a look at the user list container component:

```
export type SortOrder = "asc" | "desc";
function UsersContainer() {
  const [users, setUsers] = useState<string[]>([]);
  const [search] = useSearchParams();
  useEffect(() => {
    const order = search.get("order") as SortOrder;
    fetchUsers(order).then((users) => {
      setUsers(users);
    });
  }, [search]);
  return <Users users={users} />;
}
```

This component looks for either of the `order` query strings. It uses this as an argument to the `fetchUsers()` API to determine the sort order.

Here's what the `Users` component looks like:

```
type UsersProps = {
  users: string[];
};
function Users({ users }: UsersProps) {
  return (
    <ul>
```

```
{users.map((user) => (  
  <li key={user}>{user}</li>  
))}  
</ul>  
);  
}
```

Here is what's rendered when you navigate to `/`:

- User 1
- User 2
- User 3

Figure 7.6: Rendering the user list in default order

If you include the `order` query parameter by navigating to `/?order=desc`, here's what you get:

- User 3
- User 2
- User 1

Figure 7.7: Rendering the user list in descending order

In this section, you learned about parameters in routes.

Perhaps the most common pattern is to have the ID of a resource in your app as part of the URL, which means that components need to be able to parse out this information in order to interact with the API. You also learned about query parameters in routes, which are useful for dynamic content, filtering, or passing temporary data between components. Next, you'll learn about link components.

Using link components

In this section, you'll learn how to create links. You might be tempted to use the standard `<a>` elements to link to pages controlled by `react-router`. The problem with this approach is that these links, in simple terms, will try to locate the page on the backend by sending a `GET` request. This isn't what you want because the route configuration is already in the app and we can handle routes locally.

First, you'll see an example that illustrates how `<Link>` components behave somewhat like `<a>` elements, except that they work locally. Then, you'll see how to build links that use URL parameters and query parameters.

Basic linking

The idea of links in React apps is that they point to routes that point to components, which render new content. The `Link` component also takes care of the browser history API and looks up route-component mappings. Here's an application component that renders two links:

```
function Layout() {  
  return (  
    <>  
      <nav>  
        <p>  
          <Link to="first">First</Link>  
        </p>  
        <p>  
          <Link to="second">Second</Link>  
        </p>  
      </nav>  
      <main>  
        <Outlet />  
      </main>  
    </>  
  );  
}  
  
const router = createBrowserRouter([  
  {  
    path: "/",  
    element: <Layout />,  
    children: [  
      {
```

```
    path: "/first",
    element: <First />,
  },
  {
    path: "/second",
    element: <Second />,
  },
],
});
function App() {
  return <RouterProvider router={router} />;
}
```

The `to` property specifies the route to activate when clicked. In this case, the application has two routes: `/first` and `/second`. Here is what the rendered links look like:

[First](#)

[Second](#)

Figure 7.8: Links to the first and second pages of the app

When you click the **First** link, the page content changes to look like this:



Figure 7.9: The first page when the app is rendered

Now that you can use `Link` components to render links to basic paths, it's time to learn about building dynamic links with parameters.

URL and query parameters

Constructing the dynamic segments of a path that is passed to `<Link>` involves string manipulation. Everything that's part of the path goes to the `to` property. This means that you have to write more code to construct the string, but it also means less behind-the-scenes magic happening in the router.

Let's create a simple component that will echo back whatever is passed to the echo URL segment or the echo query parameter:

```
function Echo() {  
  const params = useParams();  
  const [searchParams] = useSearchParams();  
  return <h1>{params.msg || searchParams.get("msg")}</h1>;  
}
```

In order to get search parameters that were passed to a route, you can use the `useSearchParams()` hook, which gives you a `URLSearchParams` object. In this case, we can call `searchParams.get("msg")` to get the parameter we need.

Now, let's take a look at the `App` component that renders two links. The first will build a string that uses a dynamic value as a URL parameter. The second will use `URLSearchParams` to build the query string portion of the URL:

```
const param = "From Param";  
const query = new URLSearchParams({ msg: "From Query" });  
export default function App() {  
  return (  
    <section>  
      <p>  
        <Link to={`echo/${param}`}>Echo param</Link>  
      </p>  
      <p>  
        <Link to={`echo?${query.toString()}`}>Echo query</Link>  
      </p>  
    </section>  
  )  
}
```



```
);  
}
```

Here's what the two links look like when they're rendered:

[Echo param](#)

[Echo query](#)

Figure 7.10: Different types of link parameters

The **Param** link takes you to `/echo/From%20Param`, which looks like this:

From Param

Figure 7.11: The param version of the page

The **Query** link takes you to `/echo?msg=From+Query`, which looks like this:

From Query

Figure 7.12: The query version of the page

In learning about the `Link` component and dynamic link construction, you've unlocked a more interactive and navigable web experience, empowering users to move through your application with URL and query parameters that enrich their journey.

Summary

In this chapter, you learned about routing in React applications. The job of a router is to render content that corresponds to a URL. The `react-router` package is the standard tool for this job. You learned how routes are JSX elements, just like the components they render. Sometimes, you need to split routes into feature-based modules. A common pattern for structuring page content is to have a parent component that renders the dynamic parts as the URL changes. Then, you learned how to handle the dynamic parts of URL segments and query strings. You also learned how to build links throughout your application using the `<Link>` element.

Understanding routing in React applications lays the groundwork for building complex applications with efficient navigation, preparing you for the subsequent chapters that delve into performance optimization, state management, and integrating external APIs, ensuring a seamless user experience.

In the next chapter, you'll learn how to split your code into smaller chunks using lazy components.